

Abstraction & Specification — Answers

Q1. Write a Java interface and implement it in a class.

An interface in Java specifies what a class must do, without specifying how — a pure form of abstraction and specification.

```
// Interface: specifies WHAT a Shape must be able to do
interface Shape {
    double calculateArea();    // no body -- only the specification
    double calculatePerimeter();
}

// Class: implements HOW the specification is fulfilled, for a circle
class Circle implements Shape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }

    @Override
    public double calculatePerimeter() {
        return 2 * Math.PI * radius;
    }
}

public class InterfaceDemo {
    public static void main(String[] args) {
        Shape s = new Circle(5);
        System.out.println("Area: " + s.calculateArea());
        System.out.println("Perimeter: " + s.calculatePerimeter());
    }
}
```

The interface Shape acts as a contract/specification. Any class that implements it, such as Circle, must provide concrete definitions for every method declared in the interface. Calling code can then work with the abstract Shape type without needing to know the implementation details of Circle.

Q2. Demonstrate abstraction using access modifiers.

Access modifiers (private, protected, public, and package-private/default) support abstraction by hiding internal implementation details and exposing only what is necessary through a controlled interface.

```
public class BankAccount {
    private double balance;      // hidden internal state

    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }

    // Public method: exposed interface for depositing money
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    // Public method: exposed interface for withdrawing money,
    // with hidden validation logic
    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            return true;
        }
        return false;
    }

    // Public method: read-only access to the hidden state
    public double getBalance() {
        return balance;
    }
}
```

Here, the field balance is declared private, so it cannot be accessed or modified directly from outside the class. External code can only interact with the account through the public methods deposit, withdraw, and getBalance. This is abstraction in action: the user of the

class works with a simple, well-defined interface while the internal details and validation logic remain hidden and protected from direct interference.

Q3. Give a real-world analogy for abstraction and relate it to Java classes.

Real-world analogy: driving a car

When a person drives a car, they interact with a simple set of controls: the steering wheel, the accelerator, the brake pedal, and the gear lever. The driver does not need to know how the fuel injection system works, how the engine's internal combustion process is timed, or how the transmission physically shifts gears. All of that internal complexity is hidden; only the essential controls needed to operate the car are exposed.

Relating this to a Java class:

```
public class Car {
    private boolean engineRunning = false;
    private int speed = 0;

    // Simple exposed controls -- like the steering wheel and pedals
    public void startEngine() {
        engineRunning = true;
        System.out.println("Engine started.");
    }

    public void accelerate() {
        if (engineRunning) {
            speed += 10;
            System.out.println("Speed: " + speed);
        }
    }

    public void brake() {
        speed = Math.max(0, speed - 10);
        System.out.println("Speed: " + speed);
    }
}
```

Just as the driver only uses the pedals and wheel without worrying about the engine's internals, code that uses the Car class only calls startEngine(), accelerate(), and brake(). The private fields engineRunning and speed, along with any internal calculations, are hidden away. This is precisely what abstraction achieves in object-oriented programming: exposing a simple, essential interface while hiding unnecessary implementation detail from the user of the class.